AI Engineer Assessment: Agent-Native Order-to-Cash

The scenario

Thank you for your interest in the AI Engineer role. This take-home assessment reflects the work you would do with us: understanding a business problem, deciding what to build, directing AI agents to build it, and shipping it.

Harbor Point Supply Co. is a fictional B2B distributor of cleaning, paper, glove and takeout supplies. Its customers are restaurants, hotels, schools, clinics and gyms.

Customers email in their orders in every format imaginable: SKU lists, pasted spreadsheets, free text, forwarded threads, one-line messages from a phone. The operations team reads each email, works out who the customer is and what they want, checks stock, prices and credit, and retypes the order into the ERP. It is slow, error-prone, and the team spends its day on typing instead of on customers.

Your brief: build an agent-native way to handle this.

"Agent-native" means an AI agent does the work inside the ERP through tools you design, such as looking up customers, checking stock, drafting orders, rather than just summarizing emails. People stay in control of anything risky. Where to draw that line is one of the main decisions we want to see you make.

What you'll receive

A data pack (`harbor_point_data_pack.zip`) containing a snapshot of Harbor Point's ERP at 07:00 on Monday 2 November 2026, and the order inbox for that day:

- **ERP data** as a SQLite database and as CSV files: customers with credit limits and status, contacts, delivery addresses, the product catalog, contract prices, inventory, incoming supplier deliveries, and existing sales orders with their lines.
- **40 emails** received that day, as `.eml` files and as a JSON-lines file.
- **A README** with the business rules you can rely on (pricing, credit, discontinued products, which orders can be edited).

The emails are realistic, which means many are not clean orders. Expect to find problems we have deliberately not listed for you. Anything the README doesn't cover is a product decision for you to make and explain.

All companies, people and email addresses are fictional.

What to build and deliver

You decide the scope. A focused product that handles the important cases well beats one that tries to do everything. At a minimum, your solution should include:

- **A pipeline** that ingests the inbox and loads the ERP data into a database you control.

- **An agent** that works through tools (functions, an API or an MCP server, your choice) to read the ERP and act on it. For each email it should decide what kind of request it is and what should happen next.
- **A human review step** for anything the agent should not do on its own. You decide what falls into that category.
- **A user interface** for the operations team to see what the agent did, why, and to approve, edit or reject its proposals.
- **An audit trail** of what the agent did and which actions a person approved.
- **A live deployment** at a URL we can open, deployed through CI/CD or infrastructure as code rather than by hand.

Please send us these five things:

1. **Product brief** (1–2 pages): who the user is, the problem, what you chose to build and not build, what the agent may do on its own versus what needs approval, and how you would measure success in production.
2. **Task breakdown**: the engineering tasks you gave your AI coding agents, with the actual specs and acceptance criteria you wrote. Raw is fine; we want to see how you direct agents, not a polished document.
3. **Repository** with a README explaining how to run it locally and how it is deployed.
4. **Live URL**, plus any login we need.
5. **Results and agent log** (1 page): how your system handled the 40 emails, where it gets things wrong, where your coding agent went wrong while you built it, and how you corrected it.

Rules and timeline

- **Time:** you have 7 days from receiving this brief.
- **Use AI agents.** We expect you to build most of this with AI coding agents (Claude Code or any other harness). That is how this job is done, and how you direct and correct agents is a large part of what we assess.
- **Any stack.** Use whatever language, framework, database, cloud provider and LLM you prefer.
- **Assumptions:** where the brief or the data leaves something open, make a reasonable assumption and carry on. List your assumptions in your product brief, with a line on why you made each one.